



IDRM

Integrated Disaster Response Management

Minimum Viable Product (MVP)

A standalone modular monolith — Python 3.12 · Flask + FastAPI · PostgreSQL/PostGIS

Serene by design

Secure by default

Ready to evolve

One platform. Every stakeholder. Resilient communities.

What this deck covers

1 · Why IDRM exists

Problem, vision, stakeholders, domains, lifecycle

2 · What IDRM is

Architecture, layers, modules, flows, workflow

3 · How it is built

Tech stack, repository, database, ER model, GIS

4 · Trust & quality

Security, threat model, testing, CI/CD

5 · Running it

Ubuntu deployment, observability, disaster recovery

6 · People & future

Roles, ownership, training, evolution roadmap

01

Why IDRM Exists

Foundation — vision, problem, people, and purpose

The problem we solve

Siloed data

Fragmented systems delay situational awareness.

Slow coordination

Poor information flow hurts timely decisions.

Manual processes

Duplicate reporting reduces accuracy.

Limited visibility

Hard to allocate and track resources.

Weak last-mile

Citizens under-informed during emergencies.

Little standardisation

Poor interoperability across tools.

Where we are headed, and how

Vision

A disaster-resilient world where communities, institutions, and responders act early, respond faster, and recover stronger — saving lives and livelihoods.

Mission

To integrate people, processes, data, and technology on a unified platform that enables collaboration, situational awareness, and informed action across the disaster management lifecycle.

Six guiding principles

People-Centric

Safety, dignity, inclusion for all.

Integrity & Trust

Transparency, accountability, privacy.

Interoperability

Connect systems via open standards.

Timeliness

Early warnings, rapid response.

Innovation

Evidence-based decisions from data.

Sustainability

Strengthen long-term resilience.

Six strategic goals

1 Integrated Platform

Unify data, tools, workflows.

2 Situational Awareness

Real-time, accurate information.

3 Effective Communication

Reliable alerts & coordination.

4 Community Empowerment

Engage volunteers & citizens.

5 Data-Driven Decisions

Optimise resources & outcomes.

6 Resilient Ecosystem

Sustainable partnerships.

Scope of the MVP

In scope

- Situational awareness & dashboard
- Incident reporting & alerts
- Resource management
- Tasking & coordination
- Communication & notifications
- Reports & analytics (basic)
- User & role management
- Mobile & web access
- Data security & audit

Out of scope (for now)

- Advanced AI / ML predictions
- Complex financial management
- Long-term recovery planning
- Custom hardware solutions
- Deep third-party integrations

Who IDRM serves

Citizens & communities

Report needs, receive alerts.

Volunteers & NGOs

Register skills, take tasks.

Emergency services

Police, fire, medical, SDRF/NDRF.

District administration

Coordinate & allocate.

State & national agencies

Policy, oversight, escalation.

Line departments

Health, power, water, transport.

Partners & donors

Support and transparency.

Media & public channels

Disseminate verified updates.

Key user personas & responsibilities

Citizen

Report incidents, request help, receive alerts, give feedback.

Volunteer

Register availability, respond to tasks, share updates.

Emergency Responder

Act on alerts, execute response, share status.

Coordinator

Monitor, allocate resources, issue directives.

Administrator

Manage users, roles, security, settings.

Six business domains

Incident Management

Report, classify, track, escalate.

Resource Management

Inventory, fleet, shelters.

Volunteer Management

Register, skills, assignments.

GIS & Mapping

Maps, layers, routing, geofencing.

Communication

Email, SMS, in-app alerts.

Analytics

Reports, KPIs, dashboards.

The disaster lifecycle IDRM supports



IDRM supports the full lifecycle — not just the emergency itself — so communities prepare before and recover stronger after.

How we deliver — the SDLC roadmap

1 · Idea

Discovery, feasibility, value.

2 · PRD & Architecture

Requirements & design specs.

3 · Development

Build to standards, versioned.

4 · Testing

Functional, performance, UAT.

5 · Deployment

CI/CD, go-live, rollback.

6 · Operations

Monitor, backup, support, SLA.

7 · Continuous Improvement

Feedback, tuning, roadmap.

02

What IDRM Is

Architecture — one application, cleanly modular

A modular monolith at a glance

ENTRY POINTS

Web browser (Flask UI) · REST APIs (FastAPI) · Admin/Ops · Mobile/PWA (future) · Third-party systems

APPLICATION CORE — one Python 3.12 process

Web UI (Flask/Jinja2) · APIs (FastAPI) · Background jobs (threading · APScheduler) · Business modules · Cross-cutting (security · logging · config)

DATA & STORAGE

PostgreSQL + PostGIS (primary) · Redis (cache/sessions) · Object / file storage

The eight layers

1 · User & Interaction

2 · Core Functional

3 · Business Services

4 · Platform Services

5 · Data & Integration

6 · Intelligence & Decision Support

7 · Infrastructure & Operations

8 · Security, Governance & Compliance

What the system does

Preparedness

Risk assessment, planning, SOPs.

Early Warning

Threat detection, alerts.

Response

Incident mgmt, field ops.

Resource Management

Assets, allocation, tracking.

Volunteer Management

Onboarding, deployment.

Situational Monitoring

Dashboards, maps, KPIs.

Business modules

Auth & Access

Login, JWT, RBAC.

Incident

Create, classify, track.

Resource

Inventory, deployments.

Volunteer / Responder

Skills, assignments.

GIS & Mapping

Maps, layers, routing.

Notification / Alerts

Email, SMS, in-app.

Reports & Analytics

KPIs, dashboards.

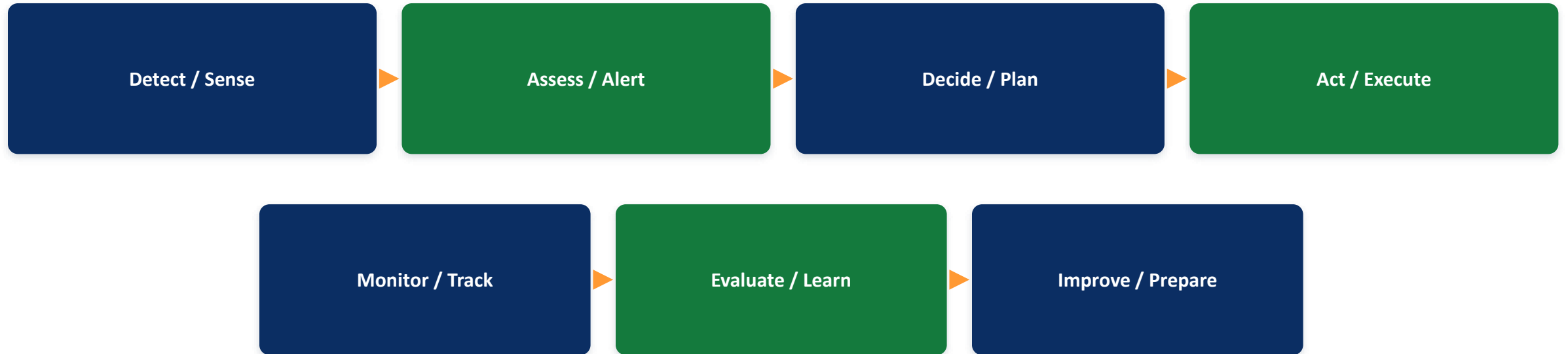
Document / Media

Uploads, attachments.

Audit & Activity

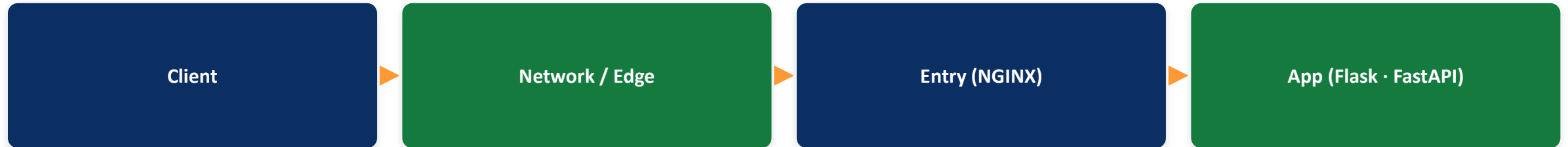
Immutable trail.

From sense to insight



Data is captured, validated, turned into situational awareness, acted on, tracked, and fed back into preparedness.

How a request is processed



Request processing pipeline

- Routing
- Middleware (CORS, auth, log)
- Authentication (JWT)
- Authorization (RBAC)
- Validation (Pydantic)
- Controller → Business → Data
- Response (serialize, headers)

End-to-end business workflow



Poster 7: information, people, and resources flow from early warning through response to recovery and continuous improvement.

03

How It Is Built

Engineering & data — the MVP stack

Technology stack (MVP) — and why

Python 3.12

One language for the whole app.

Flask + Jinja2 + Bootstrap

Server-rendered, accessible web UI.

FastAPI + Pydantic

Validated REST APIs, OpenAPI docs.

threading · APScheduler

In-process background & scheduled jobs.

SQLAlchemy + Alembic

ORM and versioned migrations.

PostgreSQL + PostGIS

Reliable data with maps built in.

Redis

Caching and sessions (optional).

GDAL (Miniconda)

Geospatial toolchain.

Gunicorn · Uvicorn · NGINX

Serve Flask & FastAPI in production.

Repository structure (monolith)

app/

The application — one codebase.

app/models · schemas

Data (SQLAlchemy) & validation (Pydantic).

app/services

Business logic per module.

app/apis · routes · templates

FastAPI, Flask views, Jinja2 pages.

app/tasks · utils

Background jobs & shared helpers.

migrations · tests · scripts

Alembic, pytest, automation.

Database architecture

PostgreSQL

The single source of truth.

PostGIS

Locations, shapes, spatial queries.

Redis

Fast cache & sessions (optional).

Object storage

Files, media, backups (S3-compatible).

Entity groups (18 tables, Poster 22)

Access & Administration

users · roles · organizations · locations

Incident Management

incidents · incident_types · incident_updates · alerts

Response Management

responders · resources · resource_types · deployments · tasks

Data & Content

documents · media · geospatial_data

Communication & Audit

communication_logs · audit_logs

Conventions

UUID keys · created_at/updated_at · PostGIS geometry

GIS architecture (MVP)

PostGIS

Stores & queries spatial data (EPSG:4326).

GDAL via Python

Reads/writes/transforms geodata — no separate map server.

Leaflet

Interactive maps in the browser.

Layers

Locations, boundaries, hazard zones, assets, routes, areas.

04

Trust & Quality

Secure by design · proven by testing

Security by design

Authentication

Login, JWT, OIDC-ready.

Authorization

Role-based access (RBAC).

Encryption

HTTPS/TLS in transit, at rest.

Input validation

Pydantic rejects bad data.

Audit trail

Immutable who-did-what.

Secure defaults

Headers, safe cookies, hardening.

Threat model

OWASP

Guards common web risks (injection, XSS, access).

STRIDE

Spoofing, Tampering, Repudiation, Info disclosure, DoS, Elevation.

Trust boundaries

Trust earned at every boundary — authenticate, validate, rate-limit.

OWASP Top 10 (2021) → key mitigations

A01 Broken Access Control

RBAC, least privilege

A02 Cryptographic Failures

TLS 1.2+, encryption, secrets mgmt

A03 Injection

Validation, parameterized queries, ORM

A04 Insecure Design

Secure SDLC, threat modeling

A05 Misconfiguration

Hardening, config mgmt, patching

A06 Vulnerable Components

Dependency scanning, updates

A07 Auth Failures

OIDC, MFA, rate limiting, logout

A08 Integrity Failures

Signing, checksums, integrity checks

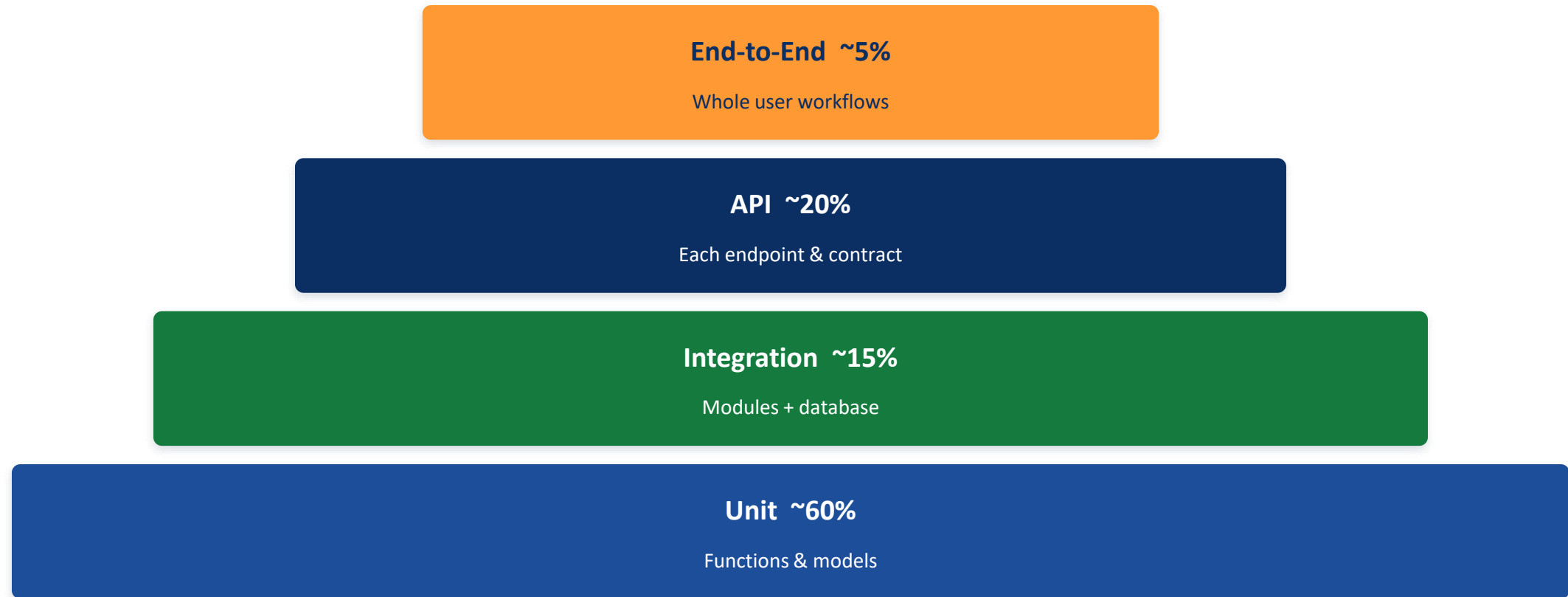
A09 Logging Failures

Central logs, alerts, retention

A10 SSRF

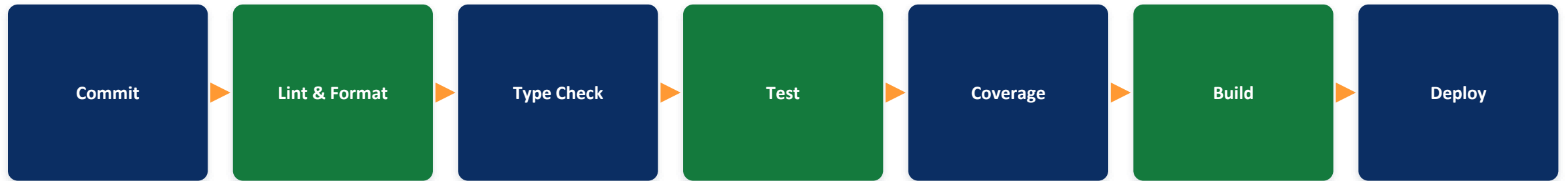
Egress filtering, URL validation

The testing pyramid



Plus: database, performance, security, contract, and user-acceptance testing. Coverage target $\approx$ 80%.

From commit to deployment



Runs on GitHub Actions. If any stage fails, the change stops there — problems are caught before users.

05

Running It

Operations on a standalone Ubuntu server

Standalone Ubuntu server

NGINX

Reverse proxy · TLS · static files

IDRM application

Gunicorn → Flask UI · Uvicorn → FastAPI · in-process jobs (systemd-managed)

Data

PostgreSQL + PostGIS · Redis · Object / file storage

Ubuntu 22.04 LTS · ~4 vCPU / 16 GB / 500 GB SSD · ports 22 / 80 / 443 only · SSH-key, Fail2Ban, Let's Encrypt · AWS is a future variant.

Seeing what the system does

Logs

Structured JSON + correlation IDs
(Loki / ELK).

Metrics

Throughput, latency, resources
(Prometheus / Grafana).

Traces

Request path & bottlenecks
(OpenTelemetry / Tempo).

Alerts

Thresholds → email / SMS /
webhook (Alertmanager).

Service-level targets (SLO / SLA)

Availability $\geq 99.5\%$ · API p95 < 800 ms · Error rate < 1% · Incident ack < 5 min · P1 resolution < 1 hr

Disaster recovery

Backup

Scheduled DB & file copies, local + off-site.

Restore

Tested restore procedure.

Failover

Standby/recovery; grows with scale.

RPO

How much recent data you can afford to lose.

RTO

How quickly you must be back up.

Resilience

A single failure never means lost data.

06

People & Future

Roles, ownership, growth, and evolution

Roles across the lifecycle

Product / Business

Vision, scope, priorities.

Architect

System design & standards.

Backend / Frontend

APIs, logic, accessible UI.

Data / GIS

Schema, migrations, maps.

Security / QA

Protection & test strategy.

DevOps / SRE

Deploy, monitor, backups.

Who owns what (RACI — one accountable owner)

APIs

Owner: API Lead · Consulted: Backend, Security, QA

Database

Owner: DBA · Consulted: Backend, Sys Admin, Security

Security

Owner: Security Eng. · Consulted: DevSecOps, Compliance

Testing

Owner: QA Manager · Consulted: Developers, Product

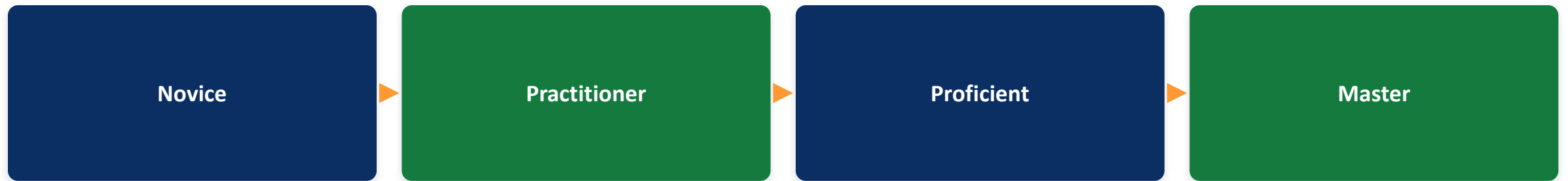
Documentation

Owner: Tech Writer · Consulted: Architect, Devs

CI/CD

Owner: DevOps · Consulted: Developers, QA, Security

Novice → mastery, for every role



Each role follows a four-rung ladder — understand the why & what, do the core work, work independently, then set direction and mentor — guided by the documentation set.

Start simple, grow with confidence — 10 years

1 · Foundation

Yr 0–1 · Core modules, RBAC & audit, maps & basic analytics, stable on-prem.

2 · Stabilization

Yr 1–2 · Performance, HA, data governance, agency integrations, DR.

3 · Integration

Yr 2–3 · API ecosystem, GIS/remote sensing, IoT feeds, data sharing.

4 · Intelligence

Yr 3–5 · AI/ML prediction, NLP, predictive alerts, decision support.

5 · Autonomy

Yr 5–7 · Workflow automation, auto-dispatch, chatbot, computer vision.

6 · Resilience 2.0

Yr 7–10 · Self-healing, cross-border interop, community resilience.

One integrated ecosystem

People

- Citizens
- Volunteers & responders
- Coordinators
- Agencies & partners

Capabilities

- Incident · Resource
- Volunteer · GIS
- Communication
- Analytics

Foundation

- One Python app
- One DB with maps
- Secure & observable
- Ready to grow

Prepared today. Protected tomorrow.

Technology in service of humanity.

Faster response

Better decisions

Efficient resources

Resilient communities

IDRM MVP — one platform, every stakeholder, resilient communities.

A

Appendix

Documentation map & standards

The documentation set

Reader set (00–08)

Orientation → Foundation → Architecture →
Engineering → Data → Security → Quality →
Operations → Organization.

Glossary (90)

Plain-language definitions.

Decisions & history (99)

ADRs, conformance record, change log.

Developer deep-dive

SRS · architecture & design · API + OpenAPI · data
model · traceability.

User guides

Citizen · Responder · Coordinator · Administrator.

Posters

The authoritative visual source.

Compliance & conformance

WCAG 2.2 AA

Accessibility for all users.

ISO/IEC 27001

Information security.

ISO 22301

Business continuity.

DPDP Act, 2023

India — data protection & privacy.